parameter is optional. Sinatra treats trailing slashes as full elements, so user and user/ are different strings, but you can make the trailing slash optional with a question mark, as in user/?

You can also include a *, which will match an arbitrary number of segments in the path, the unnamed variable will either be a block parameter, or is accessible as params["splat"].

If multiple routes match the request, the first one defined and matched is taken. You can use pass which tells Sinatra to move to the next matching route.

Instead of a string, the first argument to the path matcher can be a regular expression, in which case any path string that matches the regular expression triggers the route. If you use parenthesis to capture part of the regular expression, then the capture groups are available as arguments to the block or as params["captures"], so our route with variable example could have been written as:

web/sinatra_01/sinatra_with_regex.rb

```
get %r{/user/(\w+)} do |capture|
  [
    "<p>Hello, #{params["captures"].first}</p>",
    "<p>#{capture} has the same value</p>"
  ]
end
```

Note the use of the %r for the regex literal since the string being matched contains slashes.

After the string or regex, there're some optional keyword parameters you can also pass to a route definition that act as a condition that the request must meet to trigger the route. Perhaps the most useful of these is provides: which compares against the Accept header to match based on what media type the browser is expecting.

The Sinatra code we've seen so far is very powerful for being so short but as you build up more complex Sinatra scripts, you will probably want to go beyond a top-level script and add some classes or modularity.

Sinatra allows you to import a sinatra/base module which gives you a Sinatra::Application class that you can use the same Sinatra DSL inside, like this:

web/sinatra_01/sinatra_class.rb

```
require "sinatra/base"

class SinatraApp < Sinatra::Application
  get "/" do
    "<h1>Fly me to the moon!</h1>"
  end
end
```

There's also a Sinatra::Base, which sets fewer defaults and allows you to have more control over the Sinatra setup.

Once the Sinatra code is set up as a class, it won't run as a script. There are a few ways to get it to run, one of which is to just use Rack. We can run the file in a config.ru Rackup file as just another Rack app:

web/sinatra_01/config.ru

```
require_relative "./sinatra_class"
run SinatraApp
```